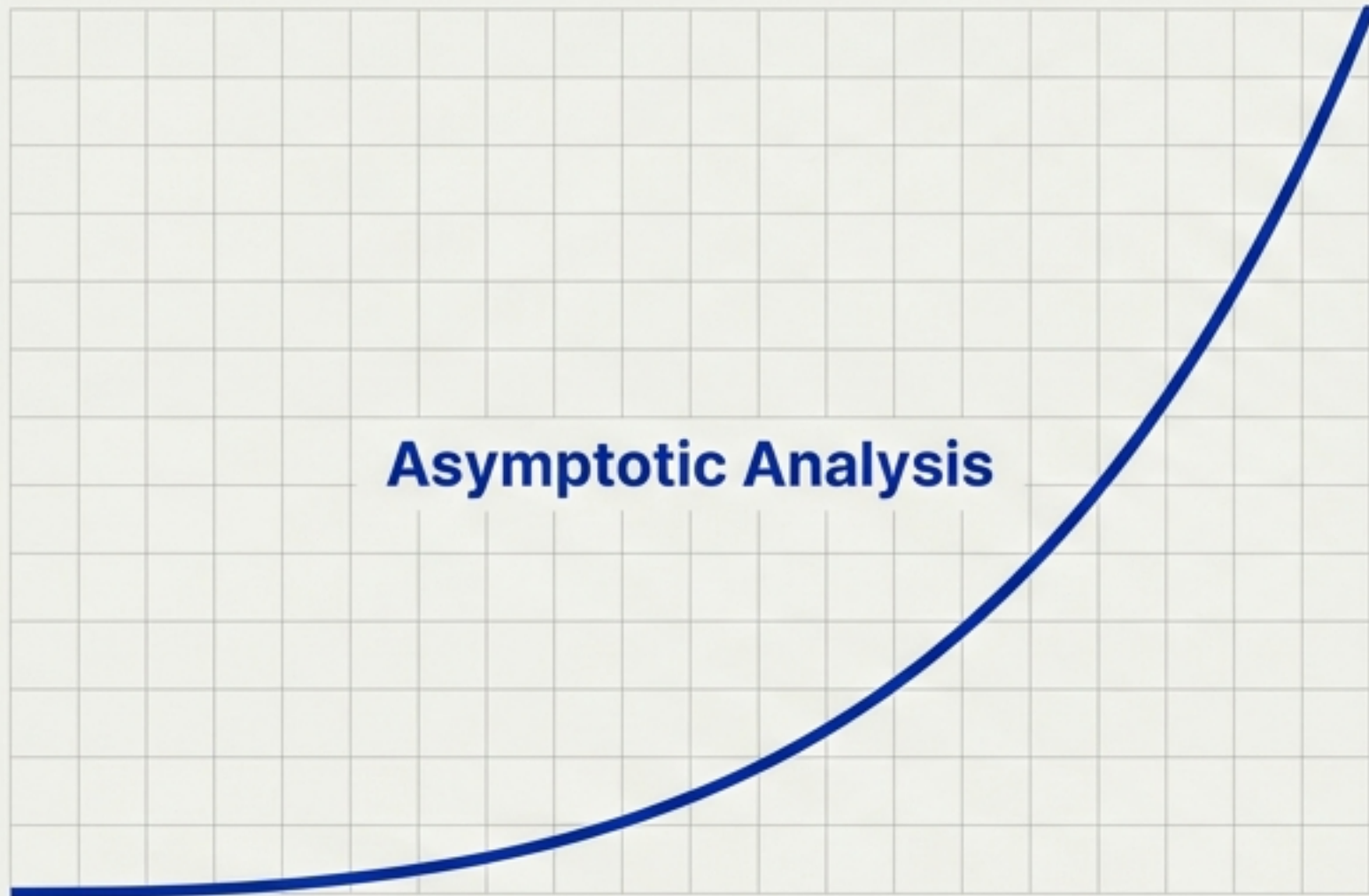


Algorithmic Mastery

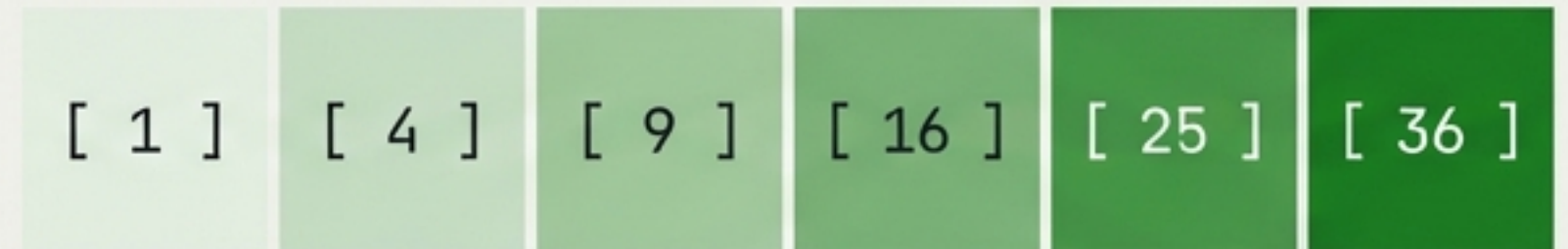
From Function Comparison to Efficient Array Partitioning



Theory: Comparison of Functions

$$f(n) = n^2$$

Implementation Strategy



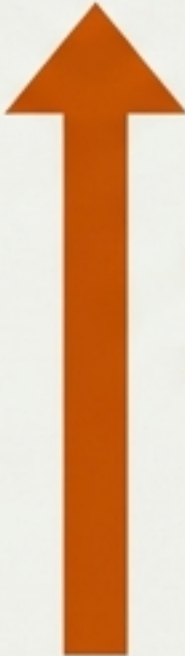
Practice: LeetCode 561 (Array Partition)

```
sort(nums); // O(n log n)
```


The Math of Efficiency: Comparing Growth Rates

To rigorously prove [Algorithm A](#) is slower than [Algorithm B](#), we compare their time complexity functions using the '[Sampling Approach](#)'. This builds intuition but lacks formal proof for infinite inputs.

Input (n)	$f(n) = n^2$	$g(n) = n^3$
2	4	8
3	9	27
4	16	64



Faster Growth /
Slower Performance

Observation: As n increases, the gap between outputs widens exponentially.

Advanced Analysis: The Logarithmic Method

For complex exponents, applying Log reduces complexity, allowing for direct term comparison.

Compare:

$$f(n) = 3n^{\sqrt{n}} \text{ vs. } g(n) = 2^{\sqrt{n} \log_2 n}$$



Apply Log

Log $f(n)$

$$\log(3) + \sqrt{n} \cdot \log(n) \approx \sqrt{n} \log n$$

Log $g(n)$

$$(\sqrt{n} \log_2 n) \cdot \log(2) = \sqrt{n} \log n$$

Result: Terms are Asymptotically Equal

The Challenge: Array Partition

LeetCode 561

Objective: Group an integer array of size $2n$ into n pairs to maximize the sum of the minimums.

Input Example:

`nums = [1, 4, 3, 2]`

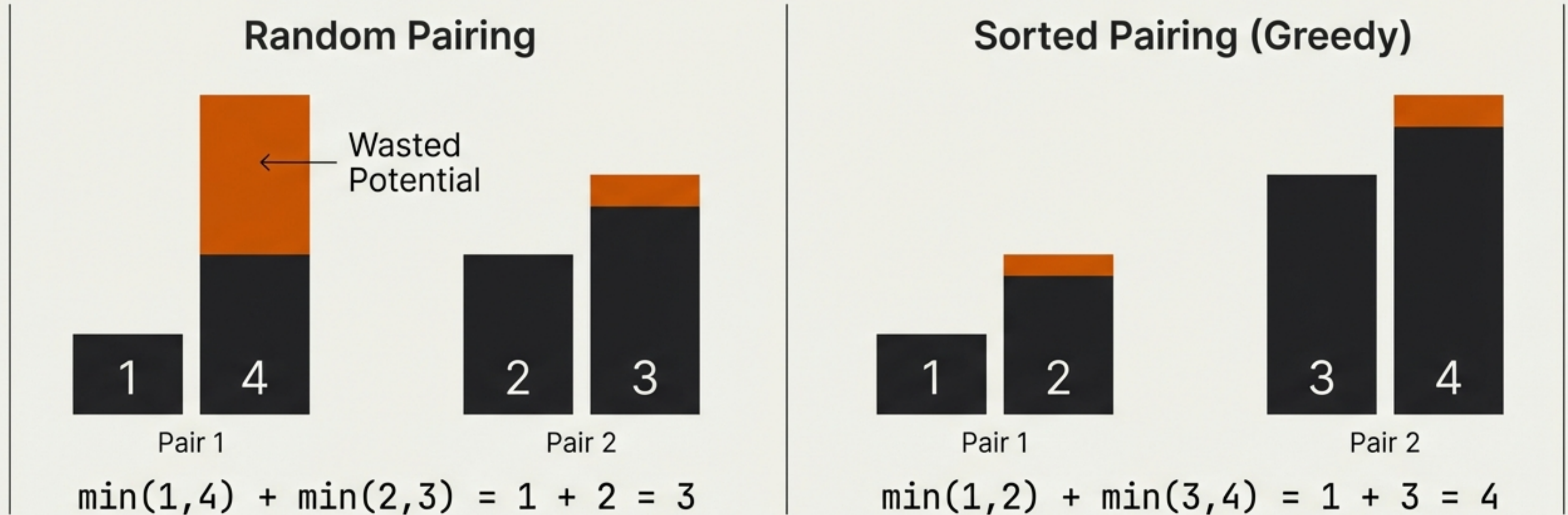
Goal: Maximize $\sum_i \min(a_i, b_i)$

Input Size N:
up to 10^4

Value Range:
-10,000 to +10,000

Constraint Note: The small value range is the key to optimization.

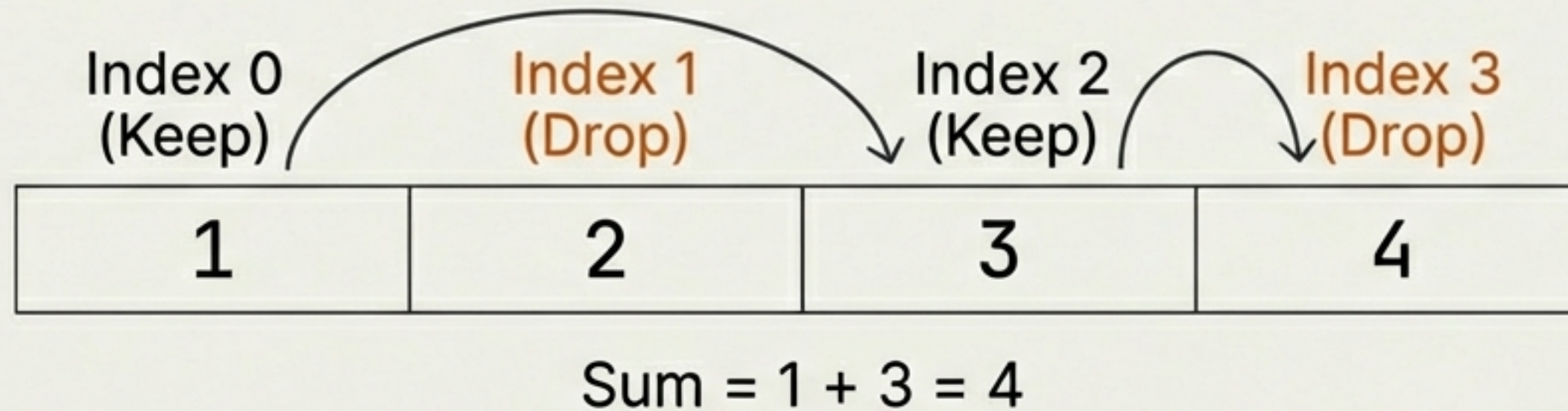
The Intuition: Don't Waste the Giants



To maximize the sum, we must sacrifice the large numbers to save other large numbers.
in Inter regular

Solution I: The Sorting Approach

Sorting the array places optimal partners adjacent to each other. We then sum elements at even indices.



```
Arrays.sort(nums);
int maxSum = 0;
for (int i = 0; i < nums.length; i += 2) {
    maxSum += nums[i];
}
return maxSum;
```


Analysis of Solution I

Time Complexity:	
$O(N \log N)$	
Space Complexity:	
$O(\log N)$	

The sorting operation (QuickSort/MergeSort) is the bottleneck. While efficient for general cases, it does not utilize the specific constraints of the problem.

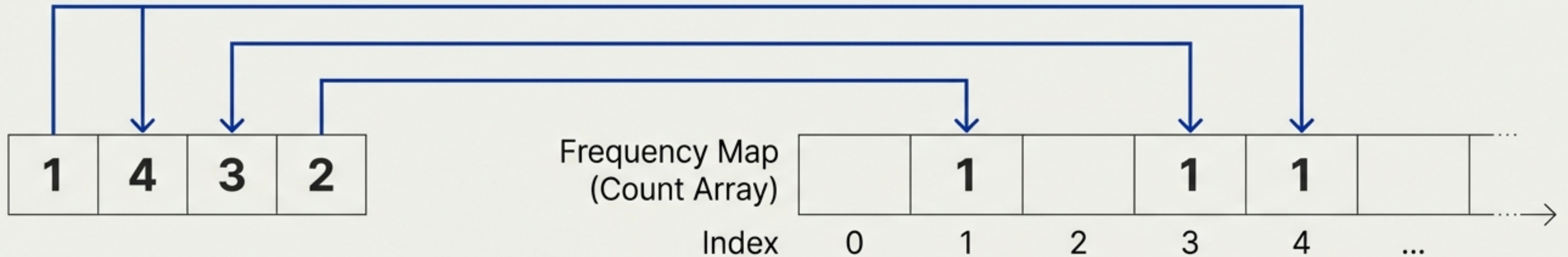
Constraint Check: The input values are limited to the range $[-10,000, 10,000]$. This small range allows for non-comparison based sorting.

Can we achieve $O(N)$?

Optimization: Leveraging Data Constraints

Input Array

Direct Addressing / Buckets



Standard Sorting:

Compares elements ($N \log N$)

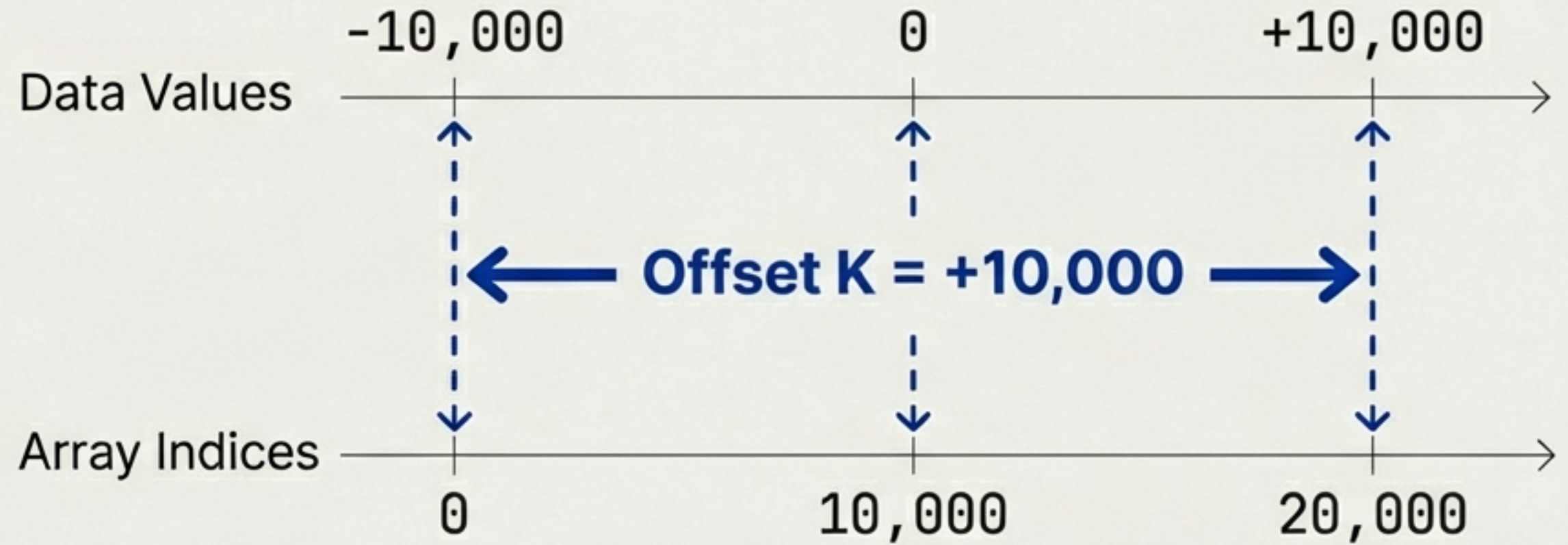
Counting Sort:

Uses value as address (N)

Range $K = 20,001$ (Constant)

The Challenge of Negative Indices

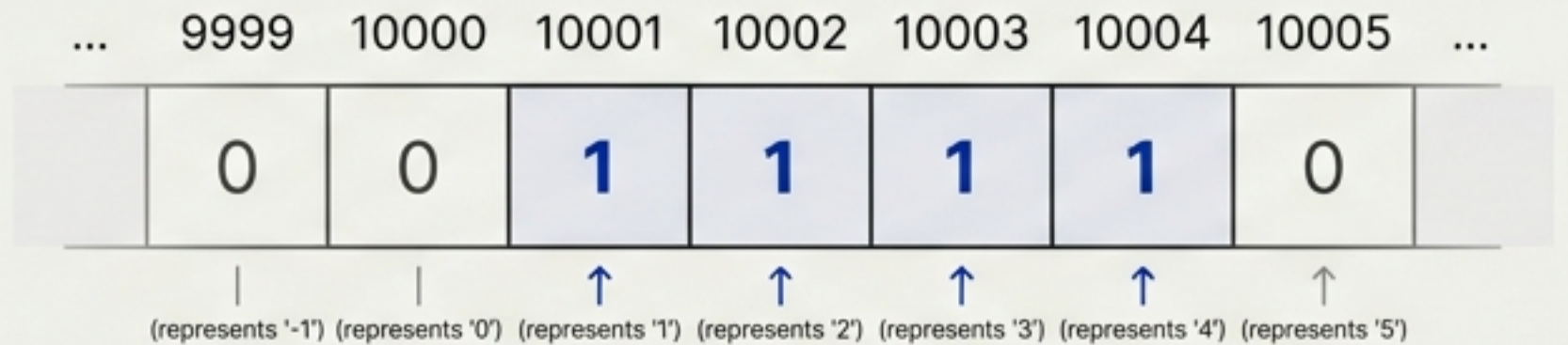
Arrays cannot have negative indices. We must apply an offset to map the data range to a valid array index.



$$\text{Index} = \text{Value} + 10,000$$

Step 1: Building the Frequency Array

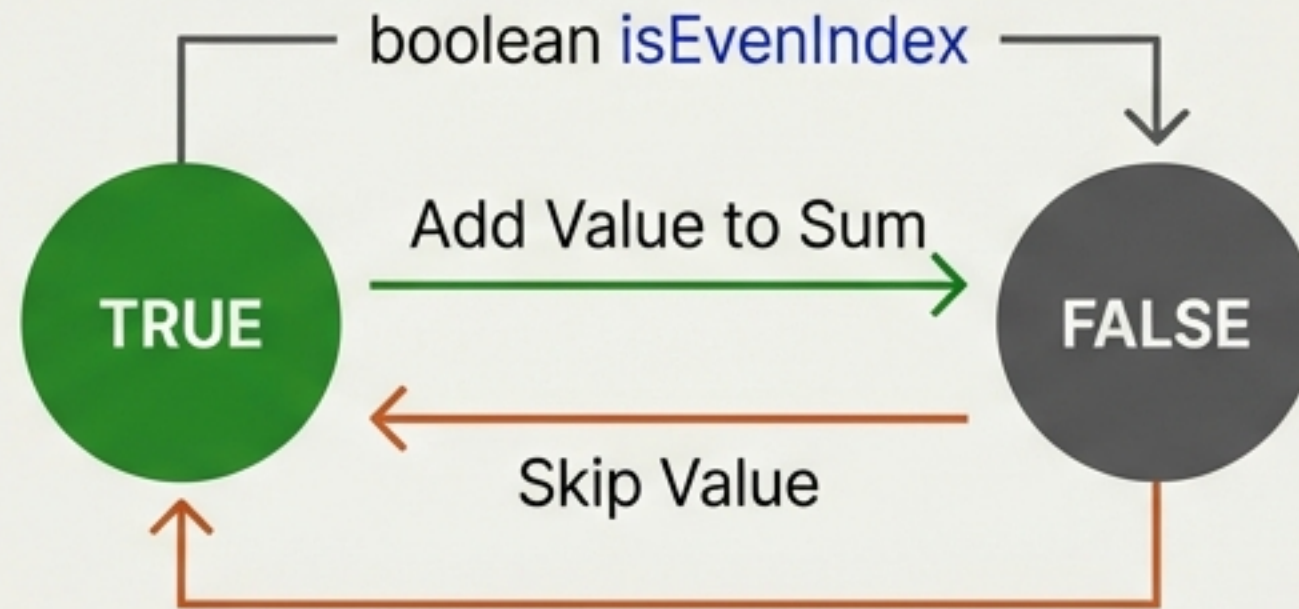
```
int[] count = new int[20001];  
int K = 10000;  
for (int num : nums) {  
    count[num + K]++;  
}
```



The frequency array naturally sorts the data as we fill it.

Step 2: The “Virtual” Traversal

Replacing Array Sorting with a Boolean Toggle

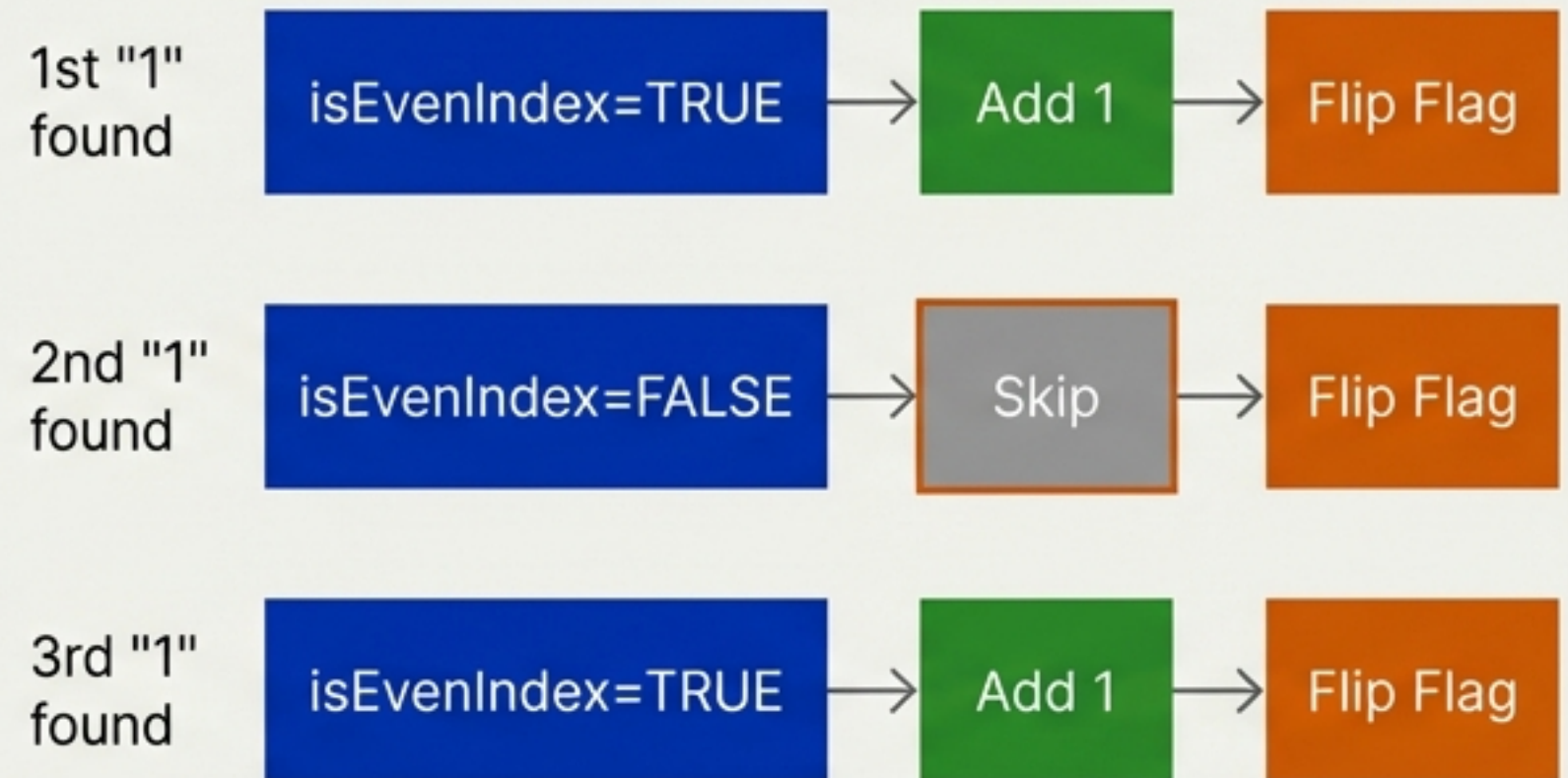


We iterate through the count array. The flag tracks if the current number belongs to an “even” (summed) index or “odd” (skipped) index in the theoretical sorted array.

Handling Duplicates with Toggle Logic

```
while (count[i] > 0) {  
    if (isEvenIndex) {  
        maxSum += (i - K);  
    }  
    isEvenIndex = !isEvenIndex;  
    count[i]--;  
}
```

Example Input: "[1, 1, 1]"



The logic correctly handles adjacent identical numbers.

Final Comparison: Sort vs. Count

Sorting Approach

- **Time: $O(N \log N)$**
- Space: $O(\log N)$
- Use Case: Unknown or large value ranges.

Counting Sort Approach

- **Time: $O(N + K) \rightarrow \underline{O(N)}$**
- Space: $O(K)$
- Use Case: Fixed, small integer ranges.

Since K (20,001) is constant, the Counting Sort solution is linear $O(N)$, making it significantly faster for this specific problem.

Summary: From Abstract Math to Optimized Code



Analyze

Use Log rules to verify asymptotic complexity.

$O(\log N)$ vs. $O(N)$



Solve

Apply Greedy strategy (Sorting) to maximize pairs.



Optimize

Exploit data constraints (Range K) to achieve $O(N)$.

$O(N + K) \rightarrow O(N)$

“True algorithmic mastery lies in understanding the constraints of your data.”